

Средства симуляции ЦП и ОС

Лекция №7

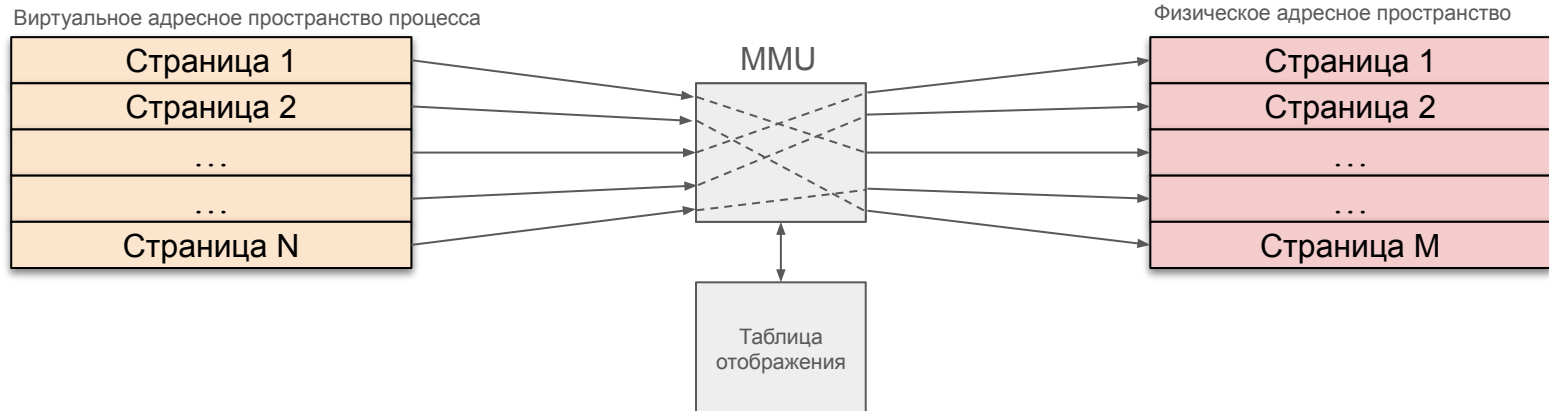


Державин Андрей
Шурыгин Антон

MMU

Проблематика отображения адресного пространства

- Виртуальная память
 - Механизм защиты адресного пространства
 - Распределение памяти между процессами
 - Оптимальность
- Реализация виртуальной памяти – совместная работа аппаратной и программной частей



Работа нескольких процессов

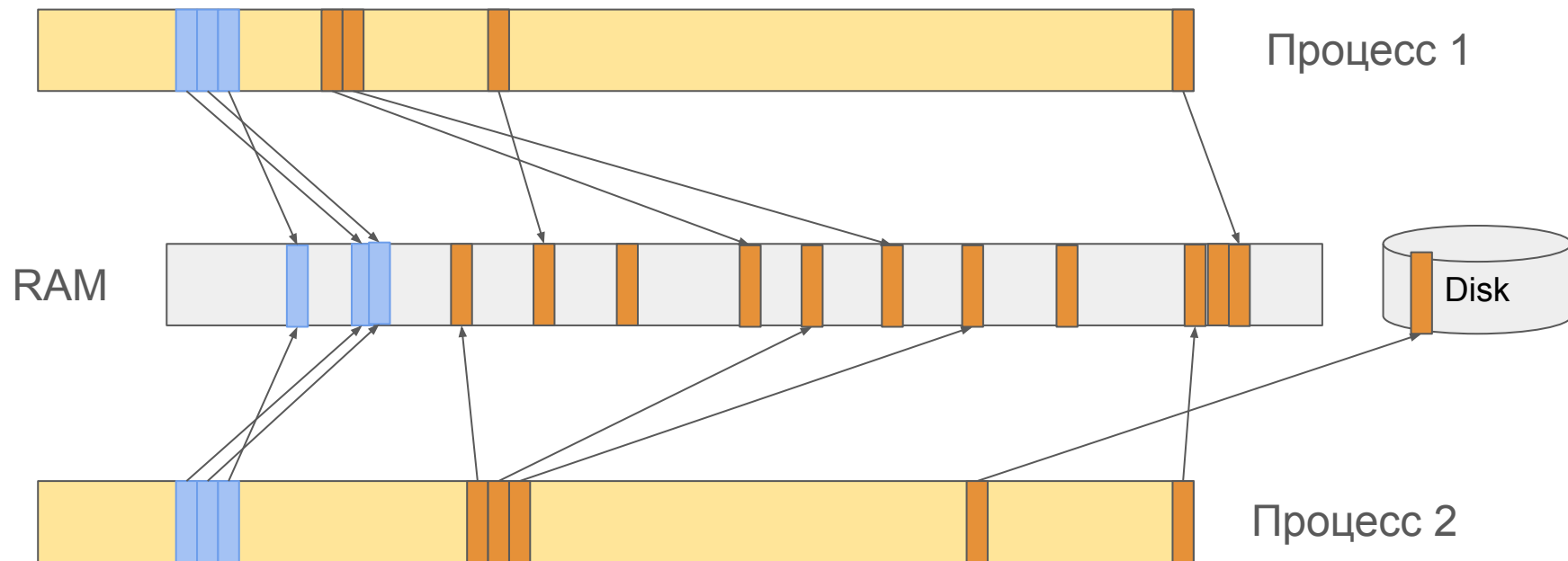


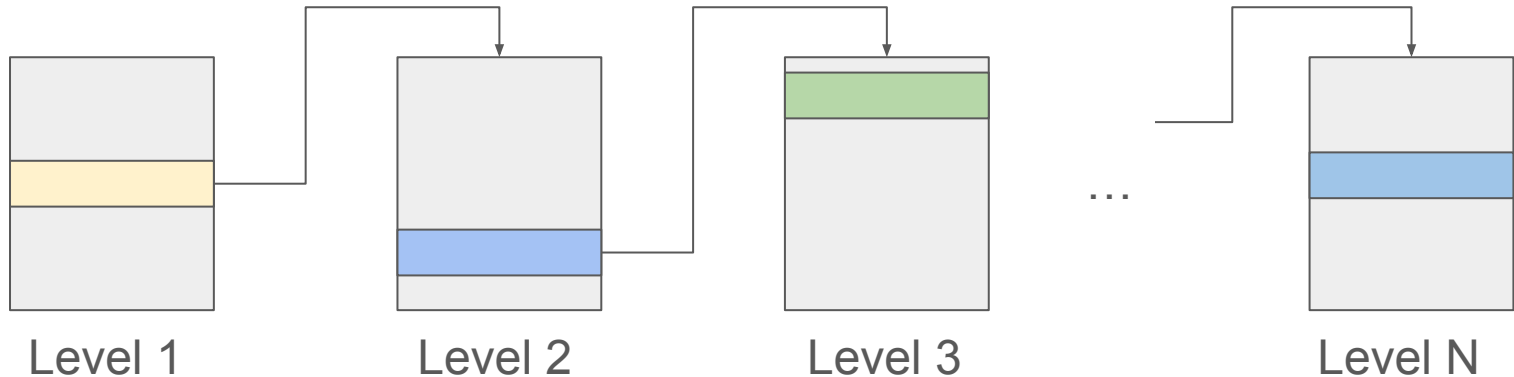
Таблица страниц

- Заполняется и поддерживается ОС
- Используется аппаратурой
- Состоит из PTE (Page Table Entry)
- Каждый процесс - своя PT

VA	PA	Present?	Permission	Dirty?
0x00000000	0x00000xxx	1	RX	0
0x00001000	0x00000xxx	0		
...				
0x00E78000	0x00007xxx	1	RW	0
0x00E79000	0x0004Dxxx	1	RW	1

Ещё таблицы?

- В современных ОС обычно применяются многоуровневые таблицы страниц
 - Чем полезно?
- Процесс трансляции - обход (walk) всех уровней таблицы



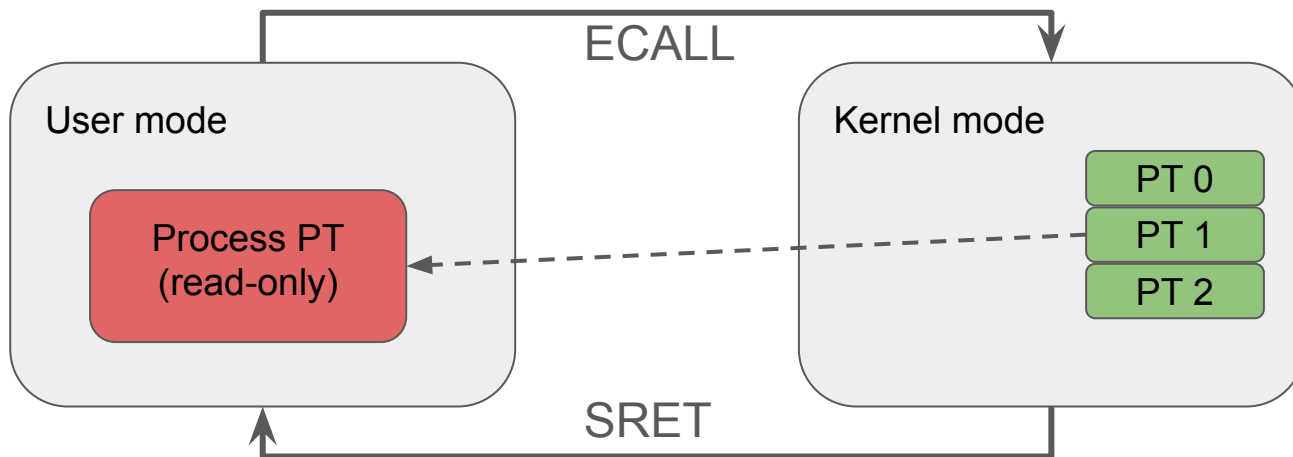
За что сегфолт?

- Segmentation fault - происходит при невалидном виртуальном адресе
 - Адрес не был аллоцирован в таблице страниц
 - Нет требуемых разрешений
- Page fault - обращение к странице, отсутствующей в физической памяти
 - ОС берет управление
 - ОС ищет свободное место в физ. памяти (Если нет места - выгружает (evicts) страницу)
 - Запрошенная страница загружается в РТ с диска (флаги обновляются)
 - ОС возвращает управление (инструкция повторяется)

```
~ via C v13.3.0-gcc  
> ./a.out  
fish: Job 1, './a.out' terminated by signal SIGSEGV (Address boundary error)
```

Защищайся

- Процессы не должны обращаться к физ. памяти друг друга
 - И в целом обращаться к PT
- Как создается PT?
 - Нужно минимум 2 режима работы CPU



Sv32

На примере

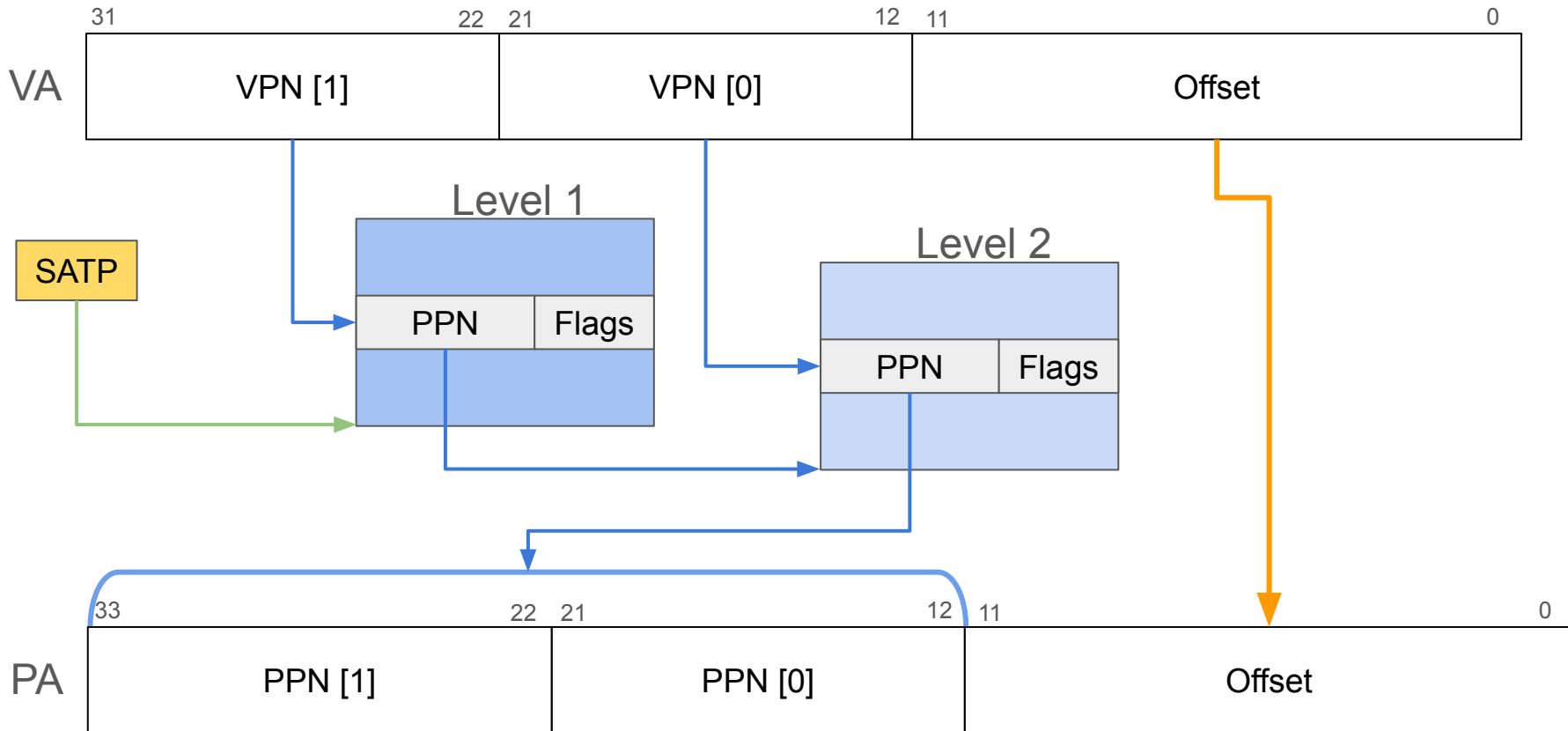
- Режимы MMU в RISC-V
 - SV32
 - 4GiB virtual memory space
 - 16GiB physical memory space
 - SV39
 - 512GiB virtual memory space
 - 32PiB physical memory space
 - SV48
 - 256TiB virtual memory space
 - 32PiB physical memory space
- 
- 32-bit systems

Sv32

● Как это работает?

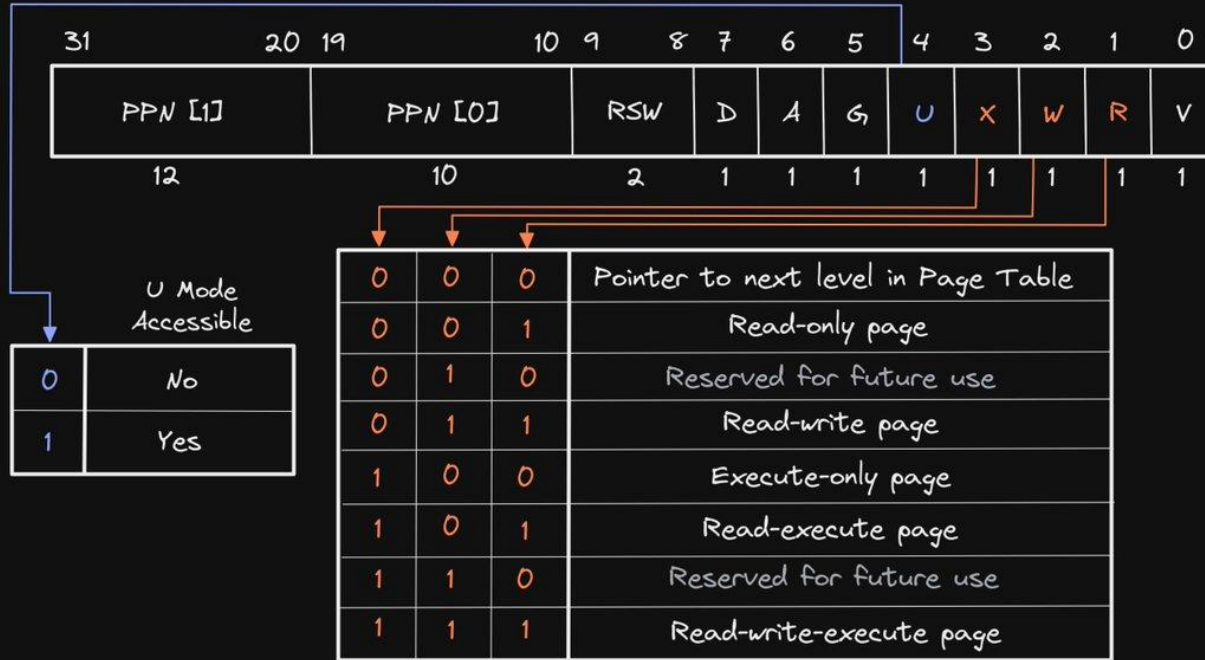
1. If XLEN equals VALEN, proceed. (For Sv32, VALEN=32.) Otherwise, check whether each bit of $va[XLEN-1:VALEN]$ is equal to $va[VALEN-1]$. If not, stop and raise a page-fault exception corresponding to the original access type.
2. Let a be $satp.ppn \times \text{PAGESIZE}$, and let $i = \text{LEVELS} - 1$. (For Sv32, $\text{PAGESIZE}=2^{12}$ and $\text{LEVELS}=2$.)
3. Let pte be the value of the PTE at address $a+va.vpn[i] \times \text{PTESIZE}$. (For Sv32, $\text{PTESIZE}=4$.) If accessing pte violates a PMA or PMP check, raise an access-fault exception corresponding to the original access type.
4. If $pte.v = 0$, or if $pte.r = 0$ and $pte.w = 1$, stop and raise a page-fault exception corresponding to the original access type.
5. Otherwise, the PTE is valid. If $pte.r = 1$ or $pte.x = 1$, go to step 6. Otherwise, this PTE is a pointer to the next level of the page table. Let $i = i - 1$. If $i < 0$, stop and raise a page-fault exception corresponding to the original access type. Otherwise, let $a = pte.ppn \times \text{PAGESIZE}$ and go to step 3.
6. A leaf PTE has been found. Determine if the requested memory access is allowed by the $pte.r$, $pte.w$, $pte.x$, and $pte.u$ bits, given the current privilege mode and the value of the SUM and MXR fields of the mstatus register. If not, stop and raise a page-fault exception corresponding to the original access type.
7. If $i > 0$ and $pte.ppn[i-1:0] \neq 0$, this is a misaligned superpage; stop and raise a page-fault exception corresponding to the original access type.
8. If $pte.a = 0$, or if the memory access is a store and $pte.d = 0$, either raise a page-fault exception corresponding to the original access type, or:
 - a. Set $pte.a$ to 1 and, if the memory access is a store, also set $pte.d$ to 1.
 - b. If this access violates a PMA or PMP check, raise an access-fault exception corresponding to the original access type.
 - c. This update and the loading of pte in step 3 must be atomic; in particular, no intervening store to the PTE may be perceived to have occurred in-between.
9. The translation is successful. The translated physical address is given as follows:
 - a. $pa.pgoff = va.pgoff$.
 - b. If $i > 0$, then this is a superpage translation and $pa.ppn[i-1:0] = va.vpn[i-1:0]$.
 - c. $pa.ppn[\text{LEVELS}-1:i] = pte.ppn[\text{LEVELS}-1:i]$.

SV32



SV32

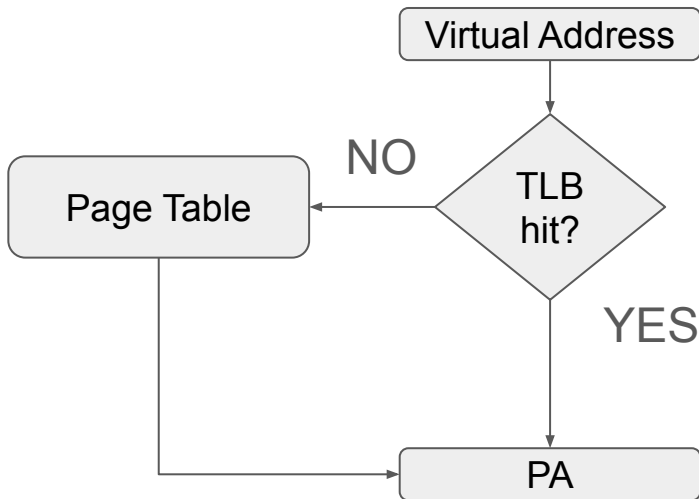
SV32 Page Table Entry (PTE) Permissions



TLB

TLB

- Каждая трансляция требует несколько доступов в память (walk)
 - Слишком много... (сколько?)
- TLB (Translation Lookaside Buffer) - кэширует использованные PTE
- Процесс трансляции ускоряется



TLB

- Возможны два варианта обработки промаха в TLB
- SW-based
 - Управление передаётся ОС
 - PTE записывается в TLB
- HW-based
 - Аппаратура отвечает за TLB miss
- Что если ОС изменяет PTE?
 - Похоже на случай SMC
 - Нужно сбросить TLB
 - `sfence.vma` в RISCV

TLB?

- Нужен ли TLB в симуляторе?
 - Да, еще для трансляций капсул
- TLB ускоряет отображение GUEST->HOST для user space
 - Похожий принцип: сохраняем результаты последних обращений

```
uint32_t value = 0;
void *host_ptr = lookupTLB(vaddr);
if (host_ptr) {
    memcpy(&value, host_ptr, sizeof(value));
} else {
    value = memory.load(vaddr);
}
```

TLBs?

- Как должен вести себя TLB при переключении контекста?
- Решение “в лоб” - очищать TLB при смене процесса
- Решение “по уму” - process id хранится как часть TLB entry
 - Одновременная трансляция для разных контекстов
 - Процессы не смогут читать TLB друг друга

Исключения

Привилегии

- ОС и пользовательские программы исполняются на одном железе
- Нужно как-то разграничить разные программы
 - ОС может изменять таблицу страниц
 - Пользовательские программы должны быть ограничены
- Для этой цели вводятся режимы привилегий



все што я
умею делоть в
этой жизни это
дышать

Пользовательская программа

Я вам

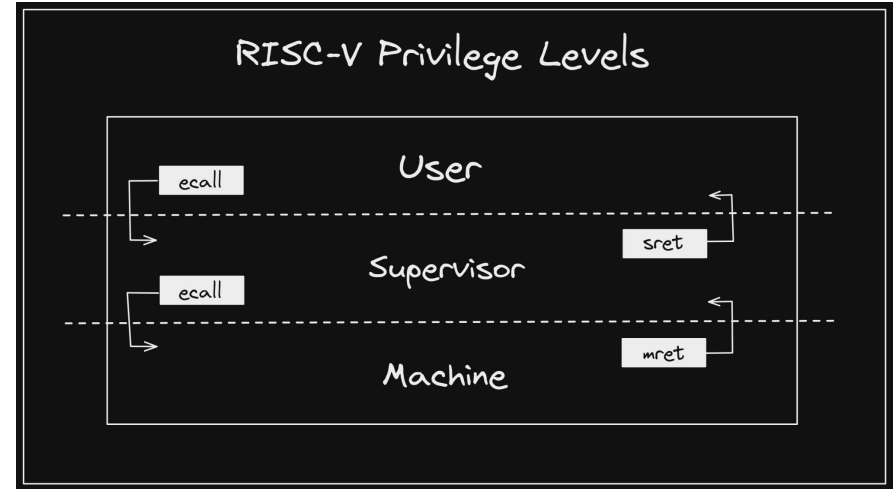
запрещаю



Операционная система

There's always a bigger fish

- M-mode (Machine)
 - Полный доступ к аппаратуре
 - Обязателен к реализации
 - Наивысший уровень привелегий
 - BIOS, прошивки...
- S-mode (Supervisor)
 - Уровень ОС
 - Управляет правилами трансляции
 - Не обязателен к реализации
- U-mode (User)
 - Уровень приложений
 - Ограниченные привелегии
 - Не обязателен к реализации



Уровни привилегий

Характеристика	Machine mode	Supervisor mode	User mode
Уровень	3	1	0
Назначение	Firmware	ОС, MMU	Пользовательские приложения
Исполнение инструкций	Все	Только U & S	Только не привилегированный
Исключения	Может обрабатывать все	Обрабатывает исключения из U	Генерирует исключения
Системные регистры	Все	Все, кроме M (только чтение)	Только чтение

Еще регистры?

- Control and Status Registers (CSR)
- Специальные регистры для управления процессором
- Отделены от GPR (x0-x31)
- В RISC-V доступно 4096 системных регистров
- Доступ **только** через специальные инструкции:
 - CSRRW, CSRRS, CSRRC, CSRRWI, CSRRSI, CSRRCI
 - CSRR (псевдоинструкция чтения)
 - CSRW (псевдоинструкция записи)
- Вид доступа (read, write) определяется уровнем привилегий
- Поля CSR могут вести себя по разному при доступе
 - WPRI - Write Preserve, Read Ignore
 - WLRL - Write Legal, Read Legal
 - WARL - Write Any, Read Legal

Виды событий

Exceptions

- Синхронные
- Связаны с исполнением
- Примеры:
 - Illegal instruction
 - ECALL
 - Page fault
 - Misaligned access
 - Segfault

Interrupts

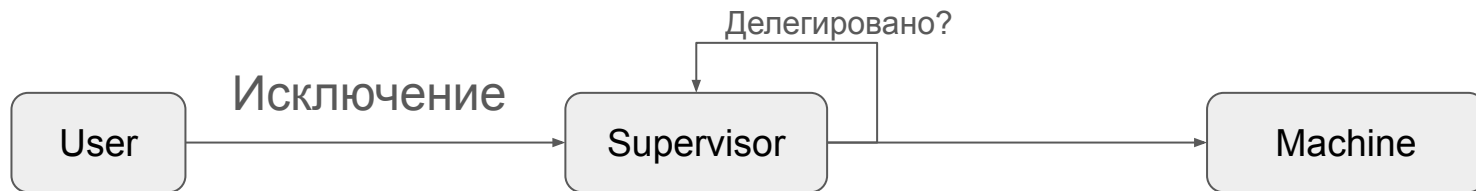
- Асинхронные
- Не связаны с исполнением
- Возникают извне
- Примеры:
 - Таймер
 - Клавиатура

Конкретика

- **X = M/S**
- **Xcause** (X Cause Register).
- **Xepc** (X Exception Program Counter)
- **Xtval** (X Trap Value)
- **Xtvec** (X Trap Vector)
- Xret - выход из обработчика
- Пример exception который вы уже реализовали?
 - ECALL!

Делегируй!

- Обработку exceptions/interrupts можно делегировать (От М к S)
- Системный регистр medeleg/mideleg
- Выставленный бит сообщает, на каком уровне обрабатывается событие
 - 1 - обрабатывается в S
 - 0 - обрабатывается в М
- ECALL обычно делегируется режиму S



Обработка исключений

- При возникновении исключения:
 - a. Сохранить PC -> mpc/sepc
 - b. Записать код причины в mcause/scause
 - c. Установить режим (M/S)
 - d. Переход на обработчик по адресу из mtvec/stvec
 - i. Либо один общий, либо таблица
- Обработчик
 - a. Анализирует mcause/scause
 - b. Выполняет действия
 - c. Выполняет mret/sret
- mret/sret:
 - a. Восстанавливает PC из mpc/sepc
 - b. Восстанавливает режим (U)

Заключение

- Разобрали MMU
 - Осуществляет трансляцию VA->PA
 - Использует таблицы страниц (многоуровневые)
 - Заполняются ОС
 - RISC-V: Sv32, Sv39...
- Ускорили трансляцию адресов с помощью TLB
 - TLB можно применять даже в симуляторе!
- Изучили какие привилегии нам даёт RISC-V
 - Режимы MSU (???)
 - Переход между режимами
- Процесс обработки исключений/прерываний
 - Его даже можно делегировать

Ссылки

1. [Simulation Foundations Book, in Russian, Grigory Rechistov](#)
2. [The RISC-V Instruction Set, Manual Volume I, Unprivileged Architecture](#)
3. [The RISC-V Instruction Set, Manual Volume II, Privileged Architecture](#)